JavaScript and React

Everything you need, explained the way a careful kid would want it. With examples.

You already understand ideas. The hard part is the words. This book gives each word a picture, then a tiny program, then the sentence people say in real work. Read it in order. JavaScript comes first, because React is JavaScript with a rule for drawing the screen.

A concept is yours when you can hide the example, write it again, and say what the screen or the number will be before you run it.

Contents

1. The four jobs on a web page
2. Values, names, and types
3. Decisions and repetition
4. Functions
5. Lists and labeled boxes
6. The page and clicks
7. Waiting without freezing
8. What React is
9. Components and JSX
10. Props: inputs to a brick
11. State: memory that redraws
12. Lists, choices, and forms
13. Effects: work after the drawing
14. How data moves
15. One small app, line by line
16. Why React feels broken
17. Glossary and a study plan
18. More of the language
19. Classes, this, and modules
20. The event loop
21. The rest of React’s memory
22. Pages, stores, style, and tests
23. Next.js and the full map
24. Interview questions and answers

1. The four jobs on a web page

A page in the browser is four jobs living together.

- **HTML** is the nouns. A heading, a button, a list. It says what exists.
- **CSS** is the clothes and the chairs. Color, size, and where things sit.
- **JavaScript** is the behavior. When someone clicks, JavaScript decides what happens next.
- **React** is a way to write that behavior so the screen is always a picture of your data. You describe the picture. React updates the page.

JavaScript also runs outside the browser, in a program called Node. The language is the same. The browser adds the page and the mouse. Node adds files and servers. This book teaches the language first, then the browser just enough, then React.

You can try a line in the browser: open any page, press F12, choose Console, and type `2 + 2`. The answer `4` is JavaScript. For React you will later use a project tool. You do not need that tool to understand the ideas.

2. Values, names, and types

A **value** is a thing the program can hold: a number, some text, yes-or-no, a list. A **name** is a sticker on that thing. In JavaScript the sticker words are `const` and `let`.

```
const name = "Priya";
let score = 9;
score = score + 1;
```

`const` means "this sticker stays on this thing." You cannot write `name = "Anu"` later. `let` means the sticker can move to a new thing. Use `const` unless you know the sticker must move. An old word, `var`, also makes stickers, but it leaks out of blocks in surprising ways. Leave it alone.

The important types:

Type	Example	Meaning
number	`3`, `1.5`	A number. There is one number type, not separate whole-number and decimal types.
string	`"hi"`	Text. Quotes can be single, double, or backticks.
boolean	`true`, `false`	Yes or no.
undefined	`undefined`	"Nobody put a value here."
null	`null`	"A value is here on purpose, and it is empty."
object	`{ city: "Pune" }`	A box of labeled values.
array	`[1, 2, 3]`	A list. Arrays are a kind of object.

`null` is a choice you make: "no user." `undefined` is what you get when you ask for a name that was never given a value, or a missing list slot.

Text can include a calculation by using backticks and `${ }`. That is a template string.

```
const name = "Priya";
const score = 10;
const line = `${name} scored ${score}`;
```

Compare with `===` and `!==`. Those check the value and the type. `==` tries to convert types first, so `"5" == 5` is true and `"5" === 5` is false. Use `===`. The surprise conversions are how quiet bugs start.

```
console.log(1 + "2");   // "12"  number becomes text and they join
console.log(1 + 2);     // 3
```

`console.log` prints to the console so you can see a value. It is the flashlight. It is not the product.

A `const` object can still change inside. The sticker cannot move. The stuff in the box can.

```
const user = { city: "Pune" };
user.city = "Mumbai";     // allowed
// user = { city: "Goa" }; // not allowed with const
```

3. Decisions and repetition

An `if` runs a block only when a test is true. `else if` is the next test. `else` is “none of those.”

```
let grade;
if (score >= 90) {
  grade = "A";
} else if (score >= 80) {
  grade = "B";
} else {
  grade = "keep going";
}
```

These values count as false in a test: `false`, `0`, `""`, `null`, `undefined`, and `NaN` (a number that means “not a number,” often from bad math). Everything else counts as true, including the text `"0"` and an empty list. An empty list is true. That surprises people.

A `for` loop repeats. This one walks indexes `0`, `1`, and `2`.

```
const colors = ["red", "blue", "green"];
for (let i = 0; i < colors.length; i++) {
  console.log(colors[i]);
}
```

`for...of` walks the items and is usually what you want.

```
for (const color of colors) {
  console.log(color);
}
```

`while` repeats as long as a test stays true. If nothing inside the loop makes the test false, it never ends. That is an infinite loop. The page freezes because JavaScript in the browser does one job at a time on the main path.

4. Functions

A function is a named recipe. You define it once. You call it whenever you want that result. `return` sends a value back and stops the function. No `return` means the function gives back `undefined`.

```
function clamp(n, low, high) {
  if (n < low) return low;
  if (n > high) return high;
  return n;
}
clamp(12, 0, 10);   // 10
```

An arrow function is the same idea written shorter. It is the form React uses most.

```
const clamp = (n, low, high) => {
  if (n < low) return low;
  if (n > high) return high;
  return n;
};
```

When the body is one expression, you can skip the braces and the word `return`.

```
const double = (n) => n * 2;
```

Parameters are the blanks in the recipe. Arguments are the values you fill in at the call. Extra arguments are ignored. Missing ones are `undefined`.

A function can see names outside itself. That outside memory is a **closure**. The inner function keeps using `step` even after `makeAdder` has finished.

```
function makeAdder(step) {
  return (n) => n + step;
}
const add5 = makeAdder(5);
add5(2);   // 7
```

Think of `makeAdder(5)` as a machine that was built already holding the number 5.

Functions are values. You can put one in a variable, pass it to another function, or return it. React does this all day: a component is a function, and an event handler is a function you hand to a button.

5. Lists and labeled boxes

An array is a list. The first item is index `0`. `length` is how many items. `push` adds at the end and changes the list. That change-in-place matters later, because React wants new lists, not secret edits.

```
const nums = [10, 20, 30];
nums[0];          // 10
nums.push(40);    // nums is now [10, 20, 30, 40]
```

Three methods do most list work. Each one takes a function and does not change the original list. They return something new.

```
const nums = [1, 2, 3, 4];

const doubled = nums.map((n) => n * 2);
// [2, 4, 6, 8]

const big = nums.filter((n) => n >= 3);
// [3, 4]

const found = nums.find((n) => n > 2);
// 3, the first match, or undefined
```

`map` means “turn every item into a new item, same count.” `filter` means “keep the items that pass the test.” `find` means “give me the first match.” React lists are almost always a `map`.

An object is a box of labels.

```
const book = { title: "Raft", pages: 80 };
book.title;                 // "Raft"
book["pages"];              // 80
const copy = { ...book, pages: 90 };
```

`...book` is the spread. It copies the labels into a new box. `copy` is a new object. `book` still has 80 pages. This is the safe way to “change” data in React: copy, then override the one label that changed.

You can pull labels out into names. That is destructuring.

```
const { title, pages } = book;
const [first, second] = [10, 20];
```

Arrays and objects are compared by identity, not by looking inside. Two lists with the same numbers are not `===`. They are two boxes.

```
const a = [1, 2];
const b = [1, 2];
console.log(a === b);   // false
```

6. The page and clicks

The browser turns HTML into a tree of objects called the **DOM**, the document object model. JavaScript can find a node and change it.

```
<button id="save">Save</button>
<p id="message"></p>

<script>
  const button = document.querySelector("#save");
  const message = document.querySelector("#message");
  button.addEventListener("click", () => {
    message.textContent = "Saved";
  });
</script>
```

`document.querySelector` finds the first matching piece. `addEventListener` says “when this happens, run this function.” The function is the handler. `textContent` sets the words. This is real and useful. It is also how pages become a tangle: every handler reaches into the page and edits whatever it wants. React exists so you stop doing that by hand.

You still need this picture. React’s job is to be the person who edits the DOM for you, after you describe the screen.

7. Waiting without freezing

Some work is not finished on this line: loading a user from the network, waiting one second, reading a file. If the browser waited with its arms folded, the page could not scroll. So JavaScript starts the slow job and keeps going. The result arrives later.

A **Promise** is an IOU. It will become a value, or it will fail. `async` and `await` let you write the waiting in a straight line.

```
async function loadUser(id) {
  const response = await fetch(`/users/${id}`);
  if (!response.ok) {
    throw new Error("Could not load user");
  }
  return response.json();
}
```

Read it as: “Pause this function until `fetch` finishes. Other code on the page may run while we pause. Then continue on the next line.” `fetch` is the browser’s way to ask a server for data. `response.json()` reads the answer as data.

`try` and `catch` hold a failure so the whole page does not crash.

```
async function show() {
  try {
    const user = await loadUser(3);
    console.log(user.name);
  } catch (error) {
    console.log(error.message);
  }
}
```

The mistake is to treat the IOU as the value. `loadUser(3)` without `await` is a Promise, not a user. You cannot read `.name` on it yet.

8. What React is

React is a library for building a screen out of functions. Each function returns a description of a piece of the page. When the data changes, you call the function again and get a new description. React looks at the new description and the old one and updates only the parts of the real page that differ.

The picture in your head:

1. Data sits in memory. A score is `0`.
2. A function reads that data and says “show a paragraph with 0 and a button.”
3. Someone clicks.
4. You replace the score with `1` using React’s setter, not by quiet assignment.
5. React calls the function again. Now it says “show a paragraph with 1.”
6. React changes the paragraph on the real page.

You do not hunt for the paragraph and edit it. You change the data, and the function is the rule for the picture. That rule is the whole trick. People struggle when they still try to edit the page the old way, or when they change data in a way React cannot see.

A **component** is one of those functions. A **re-render** is React calling it again. A **hook** is a function whose name starts with `use`, such as `useState`. Hooks are how a function keeps memory between calls. You call hooks at the top of the component, not inside an `if`, so React can match each memory slot to the same call every time.

9. Components and JSX

JSX looks like HTML inside JavaScript. It is not HTML. It is a way to write “this element, with these settings, with these children.” A tool turns it into function calls before the browser runs it. You write the readable form.

```
function Hello() {
  const name = "Priya";
  return <h1>Hello, {name}</h1>;
}
```

Curly braces jump from “tag land” back into JavaScript. `{name}` prints the value. A component name starts with a capital letter. `<Hello />` means “call this component.” `<h1>` means a real heading. Lowercase tags are HTML. Capitalized tags are your functions.

One component returns one parent. If you need two siblings, wrap them. An empty wrap `<>...</>` does not add an extra box on the page.

```
function Header() {
  return (
    <>
      <h1>Notes</h1>
      <p>A small list</p>
    </>
  );
}
```

The parentheses let the return stretch over several lines. Without them, JavaScript would return `undefined` and ignore the next lines. That bug looks like a blank screen.

Write a real attribute as `className`, not `class`, because `class` is a JavaScript word. Write `htmlFor` instead of `for` on a label. Close every tag. A comment inside JSX is `{/* like this */}`.

JavaScript expressions are allowed in the braces. Statements such as `if` are not. You compute the value above the return, or you use a conditional expression.

```
function Status({ ready }) {
  return <p>{ready ? "Ready" : "Loading"}</p>;
}
```

10. Props: inputs to a brick

Props are the inputs a parent passes to a child. The child reads them. The child does not change them. Think of a rubber stamp: the parent sets the words, the child presses them onto the page.

```
function Greeting({ name }) {
  return <p>Hello, {name}</p>;
}

function App() {
  return <Greeting name="Priya" />;
}
```

`{ name }` pulls `name` out of the props object. This is the same as:

```
function Greeting(props) {
  return <p>Hello, {props.name}</p>;
}
```

A prop can be any value: a number, a list, or a function.

```
function SaveButton({ onSave }) {
  return <button onClick={onSave}>Save</button>;
}
```

`onClick={onSave}` passes the function. `onClick={onSave()}` calls it immediately while drawing, which is almost never what you want. The parentheses mean “do it now.” Without parentheses, you hand over the recipe and the click does it later.

Whatever you put between an opening tag and a closing tag arrives as `children`.

```
function Card({ children }) {
  return <section className="card">{children}</section>;
}

function App() {
  return (
    <Card>
      <h2>Today</h2>
    </Card>
  );
}
```

11. State: memory that redraws

Ordinary variables forget, in a way that matters. Every re-render calls your function from the top. A `let score = 0` inside the function starts at 0 again. **State** is memory React keeps for you between those calls.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function increment() {
    setCount(count + 1);
  }

  return (
    <button onClick={increment}>
      Clicked {count} times
    </button>
  );
}
```

Read `useState(0)` as “remember a number, starting at 0.” It returns a pair: the current value, and a setter. The names `count` and `setCount` are a custom. Calling `setCount` does two things: it stores the new number, and it asks React to call `Counter` again. The button’s text changes because the function returns new text, not because you edited the button.

This does not redraw:

```
count = count + 1;          // if count were a let, still the wrong idea
// or
count++;                    // you cannot assign to count from useState
```

The screen follows the setter. If you change a value and React did not see the setter, the picture stays old. That single rule explains most “I updated it but nothing happened.”

When the next value depends on the previous one, and several updates might pile up, pass a function to the setter.

```
setCount((current) => current + 1);
```

React will give that function the latest stored count. Use this form for clicks that add one. Use `setCount(next)` when you already know the whole next value, such as clearing to 0 or storing what was typed.

State can be a list or an object. Replace the whole value with a new one. Do not `push` into the old list and expect a redraw. `push` edits the box in place, and React may look at it and think nothing changed, because the box is the same box.

```
function addTodo(text) {
  setTodos((current) => [
    ...current,
    { id: crypto.randomUUID(), text, done: false },
  ]);
}
```

The spread `...current` copies the old items into a new array, then the new item is added at the end. New array, so React can see the change.

State belongs to the component that needs to change it, or to the closest parent of the children that must share it. Putting every value in one giant state works and becomes muddy. Putting a copy of the same fact in two states lets them disagree. One fact, one home.

12. Lists, choices, and forms

To show a list, `map` data into elements. Each element needs a `key` that is stable for that item. React uses the key to match “this row last time” with “this row this time” when the list is reordered or filtered. The key goes on the element you return from `map`, and it is not a prop your component reads unless you also pass it under another name.

```
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

Use an id from the data. Using the array index as the key looks fine until you delete or reorder, and then the wrong row keeps the cursor or the checkbox. An index key is acceptable only for a list that never changes order.

Show or hide by choosing what to return. `&&` is a shortcut: when the left side is false, React renders nothing for that spot.

```
function Warning({ count }) {
  return (
    <div>
      {count === 0 && <p>Add your first task.</p>}
      {count > 0 && <p>{count} tasks</p>}
    </div>
  );
}
```

Do not use `count && ...` when `count` might be `0`. Zero is false, and React will print the `0` on the page. Compare with `===` or use a ternary.

A form field that shows state and writes state on every change is a **controlled input**. The input does not keep a secret copy. The picture and the data are the same string.

```
function NameField() {
  const [name, setName] = useState("");

  return (
    <label>
      Name
      <input
        value={name}
        onChange={(event) => setName(event.target.value)}
      />
    </label>
  );
}
```

`event.target` is the input element. `value` is what is in it right now. You store that string. On the next draw, `value={name}` puts it back. If you set `value` and forget `onChange`, the input is frozen, because every keystroke is thrown away and the old state is written back.

Submitting a form: call `event.preventDefault()` so the browser does not reload the page. Then use the state you already have.

```
function onSubmit(event) {
  event.preventDefault();
  addTodo(name);
  setName("");
}
```

13. Effects: work after the drawing

Rendering should be a pure description: same data in, same picture out, no fetching, no writing to the page by hand. Work that reaches outside that description is an **effect**: fetching, a timer, a subscription. `useEffect` runs that work after React has painted.

```
import { useEffect, useState } from "react";

function UserCard({ id }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let ignore = false;

    async function load() {
      const response = await fetch(`/users/${id}`);
      const data = await response.json();
      if (!ignore) setUser(data);
    }

    load();
    return () => {
      ignore = true;
    };
  }, [id]);

  if (user === null) return <p>Loading...</p>;
  return <p>{user.name}</p>;
}
```

The list at the end, `[id]`, means “run again when `id` changes.” An empty list `[]` means “run after the first paint only.” No list means “run after every paint,” which is usually a bug.

The function you return is a cleanup. React calls it before running the effect again, and when the component leaves the screen. The `ignore` flag matters because a slow answer for user 1 can arrive after you have already switched to user 2. Without the flag, user 1’s name would overwrite user 2.

You do not need an effect to compute something from state. If `fullName` is `first + " " + last`, compute it while rendering. An effect that only copies one state into another state is extra memory that can disagree with itself.

`useRef` is a box that survives re-renders and does not cause a re-render when you change `.current`. Use it for a timer id, or for remembering the DOM node when you truly must. Do not hide the score in a ref and then wonder why the screen is stale. The screen follows state.

14. How data moves

Data in React flows down. A parent passes props to children. Children do not reach up and edit the parent's variables. If a child must change the parent's state, the parent passes down a function.

```
function App() {
  const [query, setQuery] = useState("");
  return (
    <>
      <SearchBox query={query} onChange={setQuery} />
      <Results query={query} />
    </>
  );
}
```

Both children see the same `query` because it lives once, in `App`. `SearchBox` calls `onChange`, which is `setQuery`. That is lifting state: the state sits above everyone who needs it.

Passing a value through many layers that do not use it is prop drilling. It is clumsy and it is still correct. When many distant components need the same value, **context** is a pipe that skips the middle layers.

```
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext("light");

function App() {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={theme}>
      <Toolbar onToggle={() => setTheme(theme === "light" ? "dark" : "light")} />
    </ThemeContext.Provider>
  );
}

function Panel() {
  const theme = useContext(ThemeContext);
  return <section className={theme}>Body</section>;
}
```

`Panel` can sit deep under `Toolbar` and still read `theme`. Context is for data that is widely needed: theme, the signed-in user, the language. It is a poor place for every piece of state. A todo's text still belongs next to the list.

One-way flow is the reason bugs are findable. If the screen is wrong, look at the state that feeds it, then at the function that set it, then at the click that called that function. You do not search the whole page for someone who edited a paragraph.

15. One small app, line by line

This is a task list. You can type a task, add it, and mark it done. Every idea from the React chapters is in here, and nothing else. Read it once as a story, then once as code.

```
import { useState } from "react";

function App() {
  const [text, setText] = useState("");
  const [tasks, setTasks] = useState([]);

  function addTask(event) {
    event.preventDefault();
    const title = text.trim();
    if (title === "") return;
    setTasks((current) => [
      ...current,
      { id: crypto.randomUUID(), title, done: false },
    ]);
    setText("");
  }

  function toggleTask(id) {
    setTasks((current) =>
      current.map((task) =>
        task.id === id ? { ...task, done: !task.done } : task
      )
    );
  }

  const remaining = tasks.filter((task) => !task.done).length;

  return (
    <>
      <h1>Tasks</h1>
      <form onSubmit={addTask}>
        <input
          value={text}
          onChange={(event) => setText(event.target.value)}
          placeholder="Add a task"
        />
        <button>Add</button>
      </form>
      <p>{remaining} left</p>
      <ul>
        {tasks.map((task) => (
          <li key={task.id}>
            <label>
              <input
                type="checkbox"
                checked={task.done}
                onChange={() => toggleTask(task.id)}
              />
              {task.title}
            </label>
          </li>
        ))}
      </ul>
    </>
  );
}

export default App;
```

What each piece is doing:

- `text` is the draft in the box. It changes on every keystroke. It is not a task yet.
- `tasks` is the list. It starts empty.

- `addTask` stops the page reload, ignores blank text, and stores a new array that ends with the new task. Then it clears the box.
- `toggleTask` builds a new array. The matching task is a new object with `done` flipped. The other tasks are the same objects, which is fine, because they did not change.
- `remaining` is calculated while drawing. It is not a third piece of state. It cannot drift away from the list.
- The checkbox uses `checked={task.done}`, so the screen is a picture of the data. Clicking calls `toggleTask`, which changes the data, which draws the screen again.

A project created with Vite gives you a file `src/App.jsx` where this function lives, and a `main.jsx` that starts React:

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.querySelector("#root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

`#root` is an empty element in `index.html`. React draws `App` inside it. `StrictMode` in development renders twice on purpose, so accidental side effects show themselves. That double call is not a bug in your counter.

To run it on your machine, in a new folder:

```
npm create vite@latest tasks -- --template react
cd tasks
npm install
npm run dev
```

Replace the contents of `src/App.jsx` with the task list. The terminal prints a local address, usually `http://localhost:5173`. Open it. The browser has the page. You do not need the internet after the install has finished, until you want another library.

The same counter in plain JavaScript

This is the version people write before React. It is correct. It is also the habit React asks you to leave.

```
let count = 0;
const button = document.querySelector("#count");
button.addEventListener("click", () => {
  count = count + 1;
  button.textContent = `Clicked ${count} times`;
});
```

The data lives in `count`. The screen is updated by a separate line that knows about the button. If a second place also shows the count, you must remember to update that place too. React's version has one line that describes the button, and the setter is the only update. Add another `{count}` anywhere in the component and it stays in step, because both spots are drawn from the same memory.

16. Why React feels broken

These are the moments that feel like React “doesn’t work.” Each one is the picture rule or the hook rule.

- **I changed the variable and the screen stayed.** You assigned a value. Call the setter from `useState`.
- **I pushed into the array and the screen stayed.** `push` edits the old array. Pass a new array to the setter, usually with `[...old, item]` or `map`.
- **The input will not type.** It has `value={text}` and the `onChange` does not call `setText`.
- **The click ran while the page was loading.** You wrote `onClick={save()}`. Pass `onClick={save}` or `onClick={() => save(id)}`.
- **A list item shows the wrong checkbox after a delete.** The `key` was the array index. Use a stable id.
- **An effect runs forever.** The effect sets state, and that state is not listed carefully, or the effect has no dependency list and sets state every time. Fetching in render, outside `useEffect`, does the same thing: render, set state, render, set state.
- **An old fetch overwrites a new one.** Ignore answers after cleanup, as in the user card.
- **The screen is blank and there is no error you noticed.** The component returned nothing, often because `return` and the JSX were on separate lines without parentheses.
- **A number 0 appears on the page.** `{count && <p>...</p>}` prints `0` when the count is zero. Use a comparison.
- **Hooks crash with “rendered more hooks than previous.”** A `useState` or `useEffect` is inside an `if`. Keep every hook at the top, before any return.
- **Two components each have a copy of the list and they disagree.** Lift the list to the parent and pass it down.

When you are stuck, say the data out loud. “The tasks array is the truth. The checkbox is a picture of `task.done`. The click must call a setter with a new array.” If you cannot point at the setter, the screen cannot change.

A handful of JavaScript slips that look like React

- `=` assigns. `===` compares. `if (count = 1)` assigns `1` and is always true.
- `const` blocks a new assignment. It does not freeze the inside of an object.
- `map` does not edit the original. If you forget to use the array it returns, it looks like nothing happened.
- Forgetting `await` leaves you holding a Promise. Reading `.name` on it is `undefined`.
- A function created inside a loop, or a handler, sees the variables from that render. That is a closure, and it is why a stale `count` shows up if you read it inside a timeout that was started long ago. The setter function form avoids that for state updates.

17. Glossary and a study plan

Word	The picture
Value	A number, text, list, or other thing the program holds
const / let	A sticker. const stays. let can be moved to a new value
Function	A recipe you can run whenever you want
Array	A list. Index 0 is the first item
Object	A box of labeled values
map / filter	Build a new list by transforming, or by keeping some items
Spread	Copy the inside of an array or object into a new one
DOM	The browser's tree of the real page
Promise	An IOU for a value that arrives later
Component	A function that returns a description of part of the screen
JSX	The HTML-looking way to write that description
Props	Inputs a parent gives a child. The child only reads them
State	Memory React keeps. Changing it with the setter redraws
Re-render	React calling your component again to get a new description
Key	A stable name so React can track a row in a list
Effect	Work that talks to the outside, run after the picture is drawn
Context	A value available to descendants without passing it through every middle component

Eight short sessions

1. In the browser console, make a `const` object, change a field, and try to reassign the name. Say which one worked.
2. Write `map` and `filter` on a list of three tasks. Predict the new arrays before you log them.
3. Build the plain JavaScript counter with a button. Watch the separate line that edits the button.
4. Start the Vite task app and get the task list on screen. Add one task. Refresh and notice the list is gone. That is state living only in memory, which is expected.
5. Mark a task done. Point at the new object created by the spread. Say why `task.done = true` would be the wrong edit.
6. Add the "remaining" count in a second place on the page. Do not add new state.
7. Add `useEffect` that logs `tasks` whenever the list changes. Then remove it if you do not need the log. Feel the difference between drawing and an effect.
8. Break the input on purpose by deleting `onChange`. See the freeze. Put `onChange` back. That feeling is you understanding controlled inputs.

You know this half when you can build the task list from a blank `App` function without looking, and when a frozen screen makes you ask "where is the setter?" before you ask "what is wrong with React?" The rest of

the map, from `reduce` and the event loop through the router, Redux, tests, and Next.js, is in the following chapters.

18. More of the language

These are the tools you meet in real files after the basics feel familiar. Each one removes a kind of clumsiness.

Defaults, rest, and safe looking

```
function greet(name = "guest") {
  return `Hello, ${name}`;
}

function total(...nums) {
  return nums.reduce((sum, n) => sum + n, 0);
}
total(1, 2, 3);   // 6
```

A default fills in when the caller passes nothing or passes `undefined`. `...nums` packs every argument into an array. That is the rest parameter.

Optional chaining `?.` stops the lookup when the value is `null` or `undefined`, instead of crashing.

```
const city = user?.address?.city;
```

If `user` is missing, or `address` is missing, `city` is `undefined`. You did not have to write three `if` statements.

`??` means "use the right side only when the left side is `null` or `undefined`." It does not treat `0` or `""` as empty. `||` does treat those as empty, which is wrong for a score of 0 or a blank-but-present nickname.

```
const score = savedScore ?? 0;
const label = nickname || "friend";
```

More list tools

```
const nums = [1, 2, 3, 4];
nums.includes(3);                       // true
nums.some((n) => n > 3);                // true, at least one
nums.every((n) => n > 0);               // true, all of them
nums.slice(1, 3);                       // [2, 3], original unchanged
nums.findIndex((n) => n === 3);         // 2
```

`slice` copies a section and leaves the list alone. `splice` edits the original list: it can delete and insert at an index. In React state, prefer `slice`, `filter`, and `map`, which return new lists. `splice` is the in-place tool, the same family as `push`.

`reduce` walks the list and carries one running result. The second argument is the starting result.

```
const total = [10, 20, 30].reduce((sum, n) => sum + n, 0);
// 60

const byId = tasks.reduce((box, task) => {
  box[task.id] = task;
  return box;
}, {});
```

Read it as “start from 0, and for each number, return the next sum.” If you can say it with `map` or `filter`, do that. Use `reduce` when the result is one combined value: a total, a grouped object, a tally.

Map, Set, and JSON

A `Set` holds unique values and answers “is this in here?” quickly. A `Map` is a dictionary whose keys can be anything, not only strings. Ordinary objects are still the right box when the labels are strings you wrote yourself. Reach for `Map` when the key is an object or when you are adding and deleting keys constantly.

```
const seen = new Set();
seen.add("a");
seen.has("a");   // true
```

JSON is text that looks like JavaScript data. Servers send it. `localStorage` stores it. `JSON.stringify` turns data into text. `JSON.parse` turns text back into data. A function cannot survive that trip. Dates become strings.

```
const text = JSON.stringify({ title: "Buy milk", done: false });
const task = JSON.parse(text);
```

Errors you raise yourself

```
function parseCount(text) {
  const n = Number(text);
  if (Number.isNaN(n)) {
    throw new Error("Count must be a number");
  }
  return n;
}
```

`throw` stops the function and jumps to the nearest `catch`. `Number("hi")` is `NaN`, and `NaN === NaN` is false, so the test is `Number.isNaN`.

Remembering in the browser

`localStorage` keeps strings for this site even after the tab closes. `sessionStorage` keeps them until the tab closes. Neither one is a database. Both are per site, visible to anyone who can use the browser on that machine, and limited to a few megabytes.

```
localStorage.setItem("tasks", JSON.stringify(tasks));
const saved = localStorage.getItem("tasks");
const tasksAgain = saved === null ? [] : JSON.parse(saved);
```

A React app that should survive refresh reads storage when state is created and writes it after the list changes. That pair is a custom hook in chapter 21. Do not put passwords or tokens you must hide into `localStorage`. Any script on the page can read it.

Timers

```
const id = setTimeout(() => {
  console.log("one second later");
}, 1000);
clearTimeout(id);

const tick = setInterval(() => {
  console.log("tick");
}, 1000);
clearInterval(tick);
```

The number is milliseconds. The function does not run inside the current recipe. It is scheduled. In React, start the timer in an effect and clear it in the cleanup, or the timer keeps firing after the component is gone.

19. Classes, this, and modules

This

`this` is “the object before the dot at the call.” It is not “the object where the function was written,” except for arrow functions.

```
const user = {
  name: "Priya",
  greet() {
    return this.name;
  },
};
user.greet();          // "Priya"
const greet = user.greet;
greet();               // undefined, nothing is before the dot
```

An arrow function does not get its own `this`. It uses the `this` of the place it was written. That is why event handlers inside React components are arrows or functions declared in the component: they see the render’s variables through a closure, and you rarely need `this` at all in modern React.

`fn.call(obj, arg)` and `fn.bind(obj)` set the object that will be `this`. You will read them in older code. You will rarely write them in a new React app.

Classes

A class is a blueprint, the same idea as in other languages. `constructor` runs when you write `new`. Methods live on the blueprint. `extends` makes a more specific blueprint. `super` calls the parent version.

```
class Vehicle {
  constructor(name) {
    this.name = name;
  }
  label() {
    return this.name;
  }
}

class Bus extends Vehicle {
  label() {
    return `Bus ${super.label()}`;
  }
}

const bus = new Bus("River");
bus.label();   // "Bus River"
```

Under the class, JavaScript still uses prototypes: if an object does not have a field, it looks on its blueprint, then on that blueprint’s parent. You can write code for years without touching `Object.getPrototypeOf`. The useful sentence is that a method is shared, and fields set on `this` belong to that one object.

React function components replaced class components for everyday UI. One place still uses a class: an error boundary, in chapter 22, because that is how React lets a parent catch a crash in a child.

Modules

A module is a file that chooses what to share. Other files import those names. The top of the file runs once, the first time it is imported.

```
// math.js
export function add(a, b) {
  return a + b;
}
export default function clamp(n) {
  return n < 0 ? 0 : n;
}

// main.js
import clamp, { add } from "./math.js";
```

The default import has no braces. Named imports have braces and must match the exported name. `export default` is one main thing per file, often a React component. Named exports are the rest. Do not make a file that both computes on import and starts the whole app. The app starts from `main.jsx`.

20. The event loop

JavaScript does one piece of your code at a time. That current piece is the call stack: the recipe on the counter. Slow browser work, such as a timer or a network request, happens off to the side. When it finishes, a callback is placed in a queue. The queue runs only when the stack is empty.

There are two queues that matter. Promise callbacks are the fast tray, called microtasks. Timers are the later tray, called macrotasks. The fast tray is emptied before the next timer runs.

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// A, then D, then C, then B
```

Say it in order. `A` prints now. The timer is scheduled, even with a delay of 0. The promise callback is scheduled on the fast tray. `D` prints now. The stack is empty. The fast tray runs, so `C`. Then the timer runs, so `B`.

This is why a long loop freezes the page: the stack never empties, so clicks and timers wait. This is why `await` gives the page a chance to breathe: it pauses that function and lets the stack clear. This is why setting state many times in one click still draws once: React waits until your event handler finishes, then renders. That waiting-until-the-end is batching.

A timeout of 0 does not mean “right now.” It means “after the current recipe and after the promise tray.”

21. The rest of React’s memory

Lazy starting state

`useState(expensive())` runs `expensive` on every render, then throws the result away after the first time. Pass a function when the first value is costly or reads storage. React calls that function only on the first render.

```
const [tasks, setTasks] = useState(() => {
  const saved = localStorage.getItem("tasks");
  return saved === null ? [] : JSON.parse(saved);
});
```

useReducer

`useState` is enough when the next value is a small edit. `useReducer` is better when many events share one piece of state and the edits have names: add, toggle, remove. A reducer is a function `(state, action) => nextState`. It must return a new value and leave the old one unchanged. `dispatch` sends an action.

```
function reducer(tasks, action) {
  switch (action.type) {
    case "add":
      return [...tasks, action.task];
    case "toggle":
      return tasks.map((task) =>
        task.id === action.id ? { ...task, done: !task.done } : task
      );
    default:
      return tasks;
  }
}

function App() {
  const [tasks, dispatch] = useReducer(reducer, []);
  function add(title) {
    dispatch({
      type: "add",
      task: { id: crypto.randomUUID(), title, done: false },
    });
  }
}
```

The component says what happened. The reducer says how the list changes. That split is the same idea Redux uses for a whole app.

useMemo and useCallback

Both remember a result until the listed inputs change. `useMemo` remembers a value. `useCallback` remembers a function. They do not make a slow program fast by magic. They skip work, and remembering has a cost too.

```
const remaining = useMemo(
  () => tasks.filter((task) => !task.done).length,
  [tasks]
);

const toggle = useCallback((id) => {
  setTasks((current) =>
    current.map((task) =>
      task.id === id ? { ...task, done: !task.done } : task
    )
  );
}, []);
```

Use them when a calculation is actually heavy, or when a child wrapped in `React.memo` should skip rendering unless its props changed. `memo` means “if the props are the same as last time, reuse the previous picture.” A new function prop every render looks like a change, which is the case for `useCallback`. For a task list of twenty items, a plain `filter` during render is the right code. Add `useMemo` when you have measured a pause, not before.

A ref that points at the page

```
function Search() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </>
  );
}
```

`inputRef.current` is the real input after React draws it. Calling `focus` is a reach into the page, which is allowed for focus, scroll, and measuring size. The text in the box still belongs in state if you need to read it for a save.

Custom hooks

A custom hook is a function whose name starts with `use` and that calls other hooks. It exists so two components can share behavior without sharing a component. This one keeps a value and mirrors it into `localStorage`.

```
function useLocalState(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved === null ? initial : JSON.parse(saved);
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}
```

Rules, with no exceptions: call hooks only at the top of a component or another hook. Call them in the same order every time. Do not call them inside a loop, an `if`, or a nested function. The name must start

with `use` so you and the linter can see the rule applies.

22. Pages, stores, style, and tests

React Router

A router picks a component from the address bar. The page does not reload. `Link` changes the address. `useParams` reads the changing part of the path. `useNavigate` changes it from code, after a save for example.

```
import { BrowserRouter, Routes, Route, Link, useParams } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/tasks">Tasks</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/tasks" element={<TaskPage />} />
        <Route path="/tasks/:id" element={<TaskDetail />} />
      </Routes>
    </BrowserRouter>
  );
}

function TaskDetail() {
  const { id } = useParams();
  return <p>Task {id}</p>;
}
```

Install it with `npm install react-router-dom`. The address `/tasks/12` renders `TaskDetail` and `id` is `"12"`, a string. One router wraps the whole app. Routes are the table of contents.

A store, and Redux

Context is a pipe for a value. A store is a stronger idea: one state tree, and the only way to change it is to send an action through a reducer. That matters when many screens edit the same data and you want a log of what happened.

Redux is that store. Redux Toolkit is the way people write it now. `createSlice` builds the actions and the reducer together. Inside those reducers you may write `state.push`. Toolkit copies the change for you, using a helper called Immer. That permission is only inside the slice. Everywhere else, you still do not mutate.

```
import { configureStore, createSlice } from "@reduxjs/toolkit";
import { Provider, useDispatch, useSelector } from "react-redux";

const tasksSlice = createSlice({
  name: "tasks",
  initialState: [],
  reducers: {
    added(state, action) {
      state.push(action.payload);
    },
  },
});

const store = configureStore({ reducer: { tasks: tasksSlice.reducer } });

function AddButton() {
  const dispatch = useDispatch();
  const count = useSelector((state) => state.tasks.length);
  return (
    <button
      onClick={() =>
        dispatch(tasksSlice.actions.added({ id: crypto.randomUUID(), title: "New" }))
      }
    >
      {count} tasks
    </button>
  );
}

function Root() {
  return (
    <Provider store={store}>
      <AddButton />
    </Provider>
  );
}
```

`useSelector` reads. `dispatch` sends an action. The component does not hold the list. Start with `useState` and lifted state. Move to a store when several distant screens must edit one shared model and passing props has become the messy part. A theme and a signed-in name can stay in context. A task list used by one page can stay in `useState`.

Style

Way	What you write	Use it when
A CSS file	`className="card"` and a stylesheet	The look is shared and simple
CSS modules	`import styles from "./Card.module.css"` then `styles.card`	You want the class name to belong to this file only
Inline style	`style={{ color: "navy" }}`	A value depends on data, such as a width
Utility classes	Tailwind classes like `className="p-4 text-lg"`	The team has agreed to style in the markup

Inline styles want objects, and names are camelCase: `backgroundColor`, not `background-color`. Pick one main way per project. Mixing all four in one component is how a button becomes impossible to restyle.

```
/* Card.module.css */
.card { padding: 12px; border: 1px solid #ccc; }
```

```
import styles from "./Card.module.css";
export function Card({ children }) {
  return <section className={styles.card}>{children}</section>;
}
```

Tests

A React test renders the component, does what a person would do, and checks the screen. It does not check the private state variable. Testing Library and Vitest are the usual pair in a Vite project.

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { expect, test } from "vitest";
import Counter from "./Counter.jsx";

test("counts clicks", async () => {
  render(<Counter />);
  const button = screen.getByRole("button", { name: "Clicked 0 times" });
  await userEvent.click(button);
  expect(screen.getByRole("button", { name: "Clicked 1 times" })).toBeInTheDocument();
});
```

The button name is the text a screen reader would use. If the test cannot find it, the button is probably unlabeled for people too. Test the behavior: click Add, see the task. Skip tests that only say a function was called, unless that function is a real edge such as `fetch`.

Error boundaries

A crash while drawing a child would blank the whole tree. An error boundary is a parent that catches that crash and shows a fallback. This is the class component you still write. Hooks cannot catch a child’s render error.

```
import { Component } from "react";

class ErrorBoundary extends Component {
  state = { failed: false };

  static getDerivedStateFromError() {
    return { failed: true };
  }

  render() {
    if (this.state.failed) return <p>Something broke.</p>;
    return this.props.children;
  }
}
```

Wrap a panel, not every button. The boundary does not catch errors in event handlers. Those still use `try/catch`. It catches bad rendering: a mistake while producing the picture.

A button people can use

Use a `<button>` for actions and a `<a>` or `<Link>` for going somewhere. Do not put a click on a `<div>`. Tie every input to a `<label>`. The testing section and a keyboard user are asking for the same thing.

23. Next.js and the full map

Next.js is a framework around React. It adds file routes, a server, and a build. In the App Router, a folder under `app` is a URL. `app/page.js` is `/`. `app/tasks/page.js` is `/tasks`.

Components are server components unless the file says `"use client"` at the top. A server component runs on the server, can read data there, and sends HTML. It cannot use `useState`, `useEffect`, or click handlers. A client component is the React you already know: state, effects, and events. Put `"use client"` on the smallest file that needs those, and leave the rest on the server.

```
// app/page.js  — server component, no state
export default function Home() {
  return <h1>Home</h1>;
}

// app/counter/page.js
"use client";
import { useState } from "react";

export default function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>{n}</button>;
}
```

Data fetching on the server can be a direct `await` in a server component. Data fetching in a client component is still an effect or a library built for that. Do not copy the server `await` into a client file. The file's first line tells you which rules apply.

Environment variables that the browser may see, in Vite, start with `VITE_` and are read as `import.meta.env.VITE_API_URL`. In Next.js the public ones start with `NEXT_PUBLIC_`. Anything without that prefix stays on the server. A secret in a client component is no longer a secret.

Where each fact lives

Fact	Home
Text currently in one input	`useState` in that form
A list one screen edits	`useState` or `useReducer` in the parent of that screen
A list many distant screens edit	A store, or context if the edits are rare
Theme, language, current user	Context, often fed once near the root
Which page the address shows	The router
A value that must survive refresh	`localStorage` or a server, mirrored into state
A timer id or an input you must focus	A ref
A heavy calculation you have measured	`useMemo`

You have the whole map

JavaScript: values, names, decisions, functions, lists, objects, the page, promises, the event loop, classes, modules, storage, and the list tools including `reduce` and `?.`.

React: components, JSX, props, state, lists, forms, effects, refs, reducer, memo, custom hooks, router, context, Redux, CSS, tests, error boundaries, and what Next.js adds with server and client components.

The order to practice is still the task list, then refresh-proof storage with the custom hook, then a second page with the router, then one test that clicks Add. The store and Next.js come after those feel ordinary. The screen is still a picture of data. The new chapters only change where that data lives and when the picture is drawn.

End of the complete JavaScript and React guide.

24. Interview questions and answers

Say the answer in a few sentences, then give the example if they ask you to go deeper. The questions follow the book. A strong answer names the rule, not a library slogan.

JavaScript

What is the difference between `const`, `let`, and `var`? `const` cannot be reassigned. `let` can. Both live in the block where they were written. `var` is function-scoped and can be seen before its line in surprising ways. Use `const` until you must reassign. A `const` object can still have its fields changed. The sticker is fixed. The inside of the box is not.

What is the difference between `==` and `===`? `===` checks value and type. `==` converts types first, so `"5" == 5` is true and `"5" === 5` is false. Use `===`.

What is `null` versus `undefined`? `undefined` means no value was put here. `null` means a value was put here on purpose and it is empty. Test missing objects with `=== null` or `== null` only when you mean both.

Which values are falsy? `false`, `0`, `""`, `null`, `undefined`, and `NaN`. An empty array is truthy. The text `"0"` is truthy.

What do `map`, `filter`, `find`, and `reduce` do? `map` builds a new list of the same length by transforming each item. `filter` builds a new list of the items that pass a test. `find` returns the first match or `undefined`. `reduce` carries one running result, such as a sum. None of them change the original list.

What is the difference between `slice` and `splice`? `slice` copies a section and leaves the array alone. `splice` inserts or deletes in the original array. React state wants the copying tools.

What do `?.` and `??` do? `user?.address?.city` stops if `user` or `address` is `null` or `undefined`. `value ?? 0` uses `0` only when `value` is `null` or `undefined`. It keeps `0` and `""`. `||` would replace those too.

Are arguments passed by value or by reference? The parameter sticker is stuck on the same object. Reassigning the parameter does not change the caller's sticker. Mutating the object, such as `push`, is visible to the caller because there is one object.

What is a closure? A function that keeps using variables from the place it was created, after that outer function has returned. `makeAdder(5)` returns a function that still sees `5`.

What is `this`? For a normal method, it is the object before the dot at the call. Pull the method off the object and `this` is lost. An arrow function does not get its own `this`. It uses the `this` of the place it was written.

What is a Promise? An IOU for a later value or a failure. `await` pauses that function until the IOU settles and lets other code run. Without `await`, you hold the Promise, not the user. `try/catch` handles the failure.

What does this print: log A, `setTimeout` 0 logs B, a resolved promise logs C, log D? A, D, C, B. A and D run now. The promise callback is a microtask and runs before the timer. A timeout of 0 still waits until the stack is empty and the promise tray is empty.

Why does a long loop freeze the page? JavaScript runs one call stack. Clicks and timers run only when that stack is empty. A long loop never gives them a turn.

What is the difference between `localStorage` and `sessionStorage`? Both store strings for this site. `localStorage` survives closing the tab. `sessionStorage` lasts until the tab closes. Objects must go through `JSON.stringify` and `JSON.parse`. Do not store secrets there.

What is a default export versus a named export? A file has one default, imported without braces. Named exports are imported with braces and the same name: `import App, { add } from "./math.js"`.

React

What is React? A library for describing the screen as a function of data. You change data with a setter. React calls the function again and updates the real page to match the new description. You do not hunt for a DOM node on each click.

What is a component? A function whose name starts with a capital letter and that returns JSX. `<Hello />` means call that function. Lowercase tags are HTML.

What is JSX? A way to write element descriptions inside JavaScript. Curly braces jump back into JavaScript. It is compiled to function calls. `class` is written `className`. A multi-line return needs parentheses, or the function returns `undefined`.

What are props? Inputs a parent passes to a child. The child reads them and does not change them. `onClick={save}` passes the function. `onClick={save()}` calls it while rendering.

What is state? Memory React keeps between renders. `useState` returns the current value and a setter. Calling the setter stores the value and schedules a re-render. A normal variable inside the function starts over on every render.

Why didn't the screen update? The data changed in a way React could not see. Either there was no setter, or the setter was given the same array or object after `push` or a field write. Pass a new array or object, usually with spread, `map`, or `filter`.

When do you use the function form of the setter? When the next value depends on the previous one, especially if several updates can pile up: `setCount((current) => current + 1)`.

What is a key? A stable id on each element returned from `map`, so React can match rows across renders. An array index breaks when the list is reordered or filtered, because the wrong row keeps the checkbox or the cursor.

What is a controlled input? The input's `value` is state, and `onChange` writes that state. The screen and the data are the same string. `value` without `onChange` freezes the box.

What is `useEffect` for? Work that reaches outside rendering: fetch, timers, subscriptions. It runs after paint. The dependency list says when to run again. The function you return is the cleanup. Do not use an effect to copy one state into another. Compute that while rendering.

Why can an old fetch overwrite a new one? The request for id `1` can finish after the user has switched to id `2`. Cleanup sets a flag, or aborts the request, so the late answer does not call the setter.

What are the rules of hooks? Call them at the top of a component or a custom hook, in the same order every render. Do not put them in a condition, a loop, or a nested function. Custom hooks start with `use`.

What is `useRef`? A box that survives renders. Changing `.current` does not redraw. Use it for a timer id or to focus an input. Do not hide the score there and expect the screen to change. The screen follows state.

What is `useReducer`? State plus a function `(state, action) => nextState`. The component dispatches named events such as add and toggle. The reducer owns the update rules. Use it when many events edit

one value.

What do `useMemo` and `useCallback` do? They remember a value or a function until the listed inputs change. They help a measured slow calculation, or a `memo` child that should skip rendering when props are unchanged. A small filter does not need them.

What is context? A way to pass a widely needed value, such as theme or the current user, without handing it through every middle component. It is a poor home for every piece of state.

What is Redux? One store for shared state. The screen reads with `useSelector` and changes state by dispatching actions. Reducers return the next state. Redux Toolkit's `createSlice` lets you write those reducers as if you mutated a draft. That permission exists only inside the slice. Start with `useState`. Move to a store when many distant screens edit the same model.

How does routing work? React Router matches the address to an element. `Link` changes the address without a full reload. `useParams` reads `:id` from the path. The value is a string.

What is an error boundary? A class component that catches a crash while a child is rendering and shows a fallback. Hooks cannot do that job. It does not catch errors inside event handlers. Those use `try/catch`.

How do you test a component? Render it, click or type the way a person would, and assert on the screen. Query by role and name. Do not assert on the private state variable.

What is a server component versus a client component in Next.js? A server component runs on the server, can await data there, and cannot use state, effects, or clicks. A client component is marked `"use client"` and is the React with hooks and events. Put that line on the smallest file that needs it.

Short coding answers

Build a counter. `useState(0)`. The button's text is `Clicked {count} times`. `onClick` calls `setCount((n) => n + 1)`.

Add an item to a list in state. `setItems((current) => [...current, item])`. Do not `push` and then set the same array.

Toggle one task. `setTasks((current) => current.map((task) => task.id === id ? { ...task, done: !task.done } : task))`.

Remove duplicates and keep order. Walk the list with a `Set` of values already returned, or use a filter with `indexOf`. A `Set` alone does not give you a React row until you spread it back to an array.

```
function unique(items) {
  return [...new Set(items)];
}
```

Group items by a field. `reduce` into an object of arrays.

```
function groupBy(items, key) {
  return items.reduce((groups, item) => {
    const name = item[key];
    groups[name] = groups[name] ?? [];
    groups[name].push(item);
    return groups;
  }, {});
}
```

Save the task list across refresh. Initialize state from `localStorage` with the function form of `useState`. Write `JSON.stringify` in an effect that depends on the list. Store strings only.

How to practice

1. Cover the answer. Say yours in under a minute.
2. Write the counter, the add, and the toggle from memory and run them.
3. Explain the print order A, D, C, B without looking.
4. Stop when a frozen input makes you say “controlled value without a setter” before you say “React is broken.”

End of the interview questions and answers.